Lab Activity #1 – Part I: Getting started with Unix and SU

Just as scratch paper is paper that you use temporarily without the plan of saving for the long term, a "scratch directory" is temporary working space, which is not backed up and which may be arbitrarily cleared by the system administrator. Create your own scratch working directory via:

```
$ cd                This takes you to your home directory
$ mkdir ESC         This creates a new directory called ESC under your home directory
```

Here ESC stands for Exploration Seismology Course is the actual folder (or directory) you wish to conduct the labs and activities. Feel free to create the directory in any other more desirable location on the Linux network, preferably designated scratch disks. If you want to save materials permanently, it is a good idea to make use of a USB storage device.

ESC directory will be your working directory in this course and most of your activities are done under this folder.

Any program that has executable permissions and which appears on the users' PATH may be run by simply typing its name on the command line. For example, if you have set your path correctly, you should be able to do the following

```
$ suplane | suxwigb  &
                     ^ this symbol, the ampersand, indicates that
                       the program is being run in background
          ^ the "pipe" symbol
```

The command line itself is the interactive prompt that the shell program is providing so that you can supply input. The proper input for a command line is an executable file, which may be a compiled program or a Unix shell script. The command prompt is saying, "Type program name here."

Try running this command with and without the ampersand &. If you run

```
$ suplane | suxwigb
```

The plot comes up, but you have to kill the plot window before you can get your commandline back, whereas

```
$ suplane | suxwigb  &
```

allows you to have the plot on the screen, and have the commandline.

To make the plot better we may add some axis labeling:

```
$ suplane | suxwigb title="suplane test pattern"
                    label1="time (s)" label2="trace number" &
```

^ Here the command is broken across a line so it will fit this page. On your screen it would be typed as one long line.

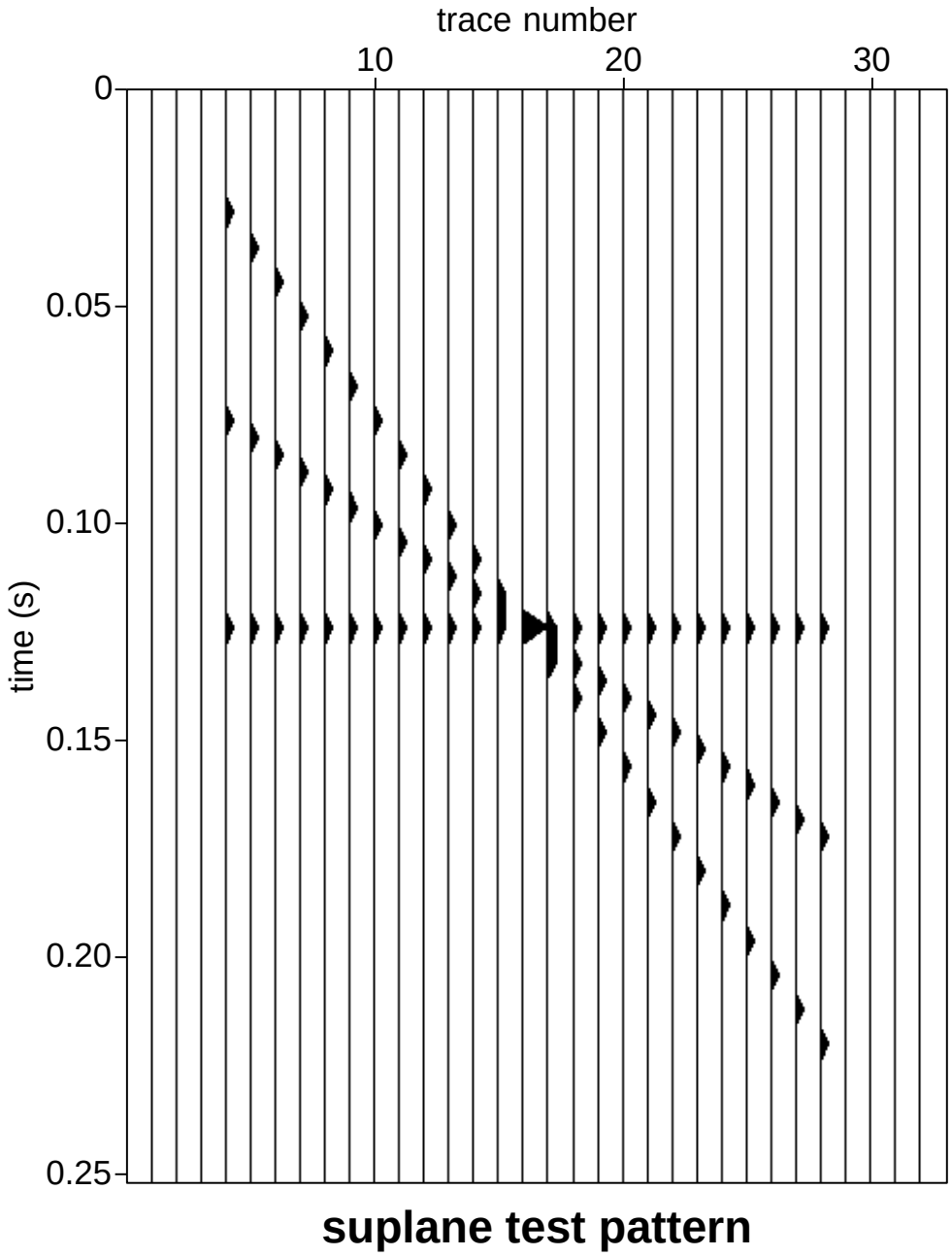

Figure 2.1: The suplane test pattern.

In Figure 2.1 you can see a test pattern consisting of three intersecting lines in the form of seismic traces. The data consist of seismic traces with only single values that are nonzero. This is *variable area* display in which each place where the trace is positive valued is shaded black. See Figure 2.1.

Equivalently, you should see the same output by typing

```
$ suplane > junk.su
$ suxwigb < junk.su  title="suplane test pattern"
                              label1="time (s)" label2="trace number" &
```

Finally, we often need to have graphical output that can be imported into documents. In SU we have graphics programs that write output in the PostScript language

```
$ supswigb < junk.su title="suplane test pattern"
                          label1="time (s)" label2="trace number" > suplane.eps
```

2.1 Pipe |, redirect in <, redirect out >, and run in background &

In the commands in the last section we used three symbols that allow files and programs to send data to each other and to send data between programs. The vertical bar | is called a "pipe" on all Unix-like systems. Output sent to standard out may be piped from one program to another program as was done in the example of

```
$ suplane | suxwigb &
```

which, in English may be translated as "run suplane (pipe output to the program) suxwigb where the & says (run all commands on this line in background)." The pipe | is a memory buffer with a "read from standard input" for an input and a "write to standard output" for an output. You can think of this as a kind of plumbing. A stream of data, much like a stream of water is flowing from the program suplane to the program suxwigb.

The "greater than" sign > is called "redirect out" and

```
$ suplane > junk.su
```

says "run suplane (writing output to the file) junk.su. The > is a buffer which reads from standard input and writes to the file whose name is supplied to the right of the symbol. Think of this as data pouring out of the program suplane into the file junk.su.

The "lest than" sign < is called "redirect in" and

```
$ suxwigb < junk.su  &
```

says "run suxwigb (reading the input from the file) junk.su (run in background).

- | = pipe from program to program

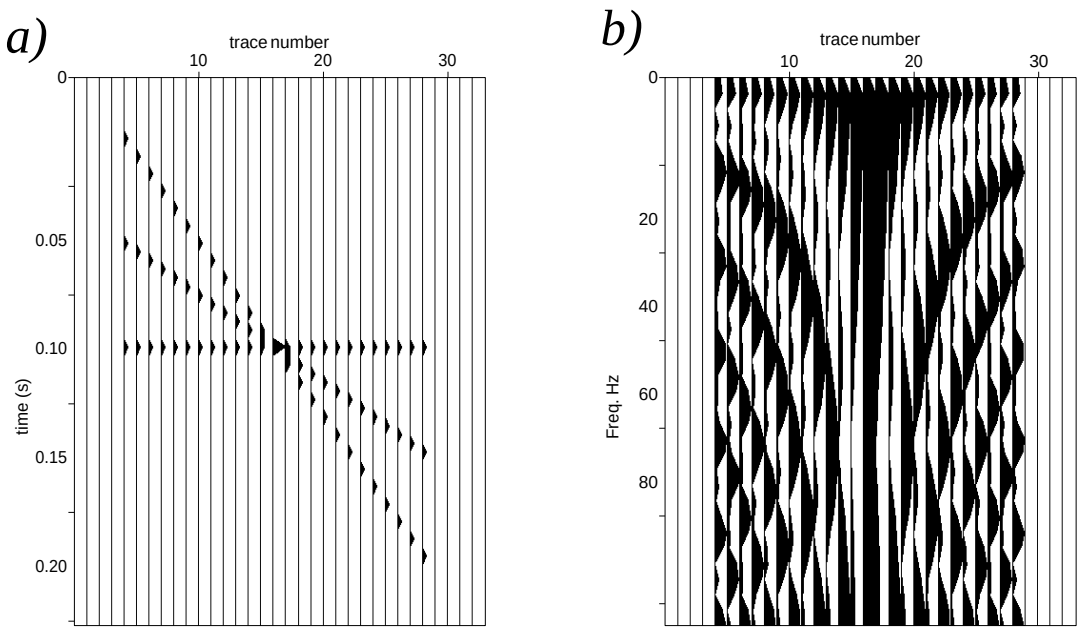

Figure 2.2
a) The suplane test pattern and b) the Fourier transform

Figure 2.2: a) The suplane test pattern. b) the Fourier transform (time to frequency) of the suplane test pattern via suspecfx.

- >= write data from program to file (redirect out)
- <= read data from file to program (redirect in)
- & = run program in background

2.2 Stringing commands together

We may string together programs via pipes (|), and input and output via redirects (>) and (<). An example is to use the program suspecfx to look at the amplitude spectrum of the traces in data made with suplane:

```
$ suplane | suspecfx | suxwigb &          --make suplane data, find the
                                           amplitude spectrum,
                                           plot as wiggle traces
```

Equivalently, we may do

```
$ suplane > junk.su                        --make suplane data, write to a file.
$ suspecfx < junk.su > junk1.su            --find the amplitude spectrum, write to
                                           a file.
$ suxwigb < junk1.su &                     -- view the output as wiggle traces.
```

This does exactly the same thing, in terms of final output as the previous example, with the exception that here, two files have been created. See Figure 2.2.

2.3 Unix Quick Reference Cards

The two figures, Fig 2.3 and Fig 2.4 are a Quick Reference cards for some Unix commands

References

Sobell, M. (2010), "A practical guide to Linux commands, editors, and shell program- ming" Pearson Education Inc., Boston, MA.

commands
card

Working with NFS files

Files saved on the UITS central Unix computers Chrome, Cobalt, Zinc, Steel, EZinfo, and STARRS/SP are stored on the Network File Server (NFS). That means that your files are really on one disk, in directories named for the central Unix hosts on which you have accounts.

No matter which of these computers you are logged into, you can get to your files on any of the others. Here are the commands to use to get to any system directory from any other system:

cd /N/u/*username*/Chrome/
cd /N/u/*username*/Cobalt/
cd /N/u/*username*/Zinc/
cd /N/u/*username*/Steel/
cd /N/u/*username*/Ezinfo/
cd /n/u/*username*/SP/

Be sure you use the capitalization just as you see above, and substitute your own username for *username*.

For example, if Jessica Rabbit is logged into her account on Steel, and wants to get a file on her EZinfo account, she would enter:

cd /N/u/jrabbit/Ezinfo/

Now when she lists her files, she'll see her EZinfo files, even though she's actually logged into Steel.

You can use the ordinary Unix commands to move files, copy files, or make symbolic links between files. For example, if John Doe wanted to move "file1" from his Steel directory to his EZinfo directory, he would enter:

mv -i /N/u/jdoe/Steel/file1

This shared file system means that you can access, for example, your Chrome files even when you are logged into Cobalt, and vice versa. However, if you are logged into Chrome, you can only use the software installed on Chrome —only users' directories are linked together, not system directories.

Figure 2.3: UNIX Quick Reference card p1. From the University References

Environment Control

Command	Description
cd *d*	*Change to directory d*
mkdir *d*	*Create new directory d*
rmdir *d*	*Remove directory d*
mv *f1 [f2...] d*	*Move file f to directory d*
mv *d1 d2*	*Rename directory d1 as d2*
passwd	*Change password*
alias *name1 name2*	*Create command alias*
unalias *name1*	*Remove command alias name1*
rlogin *nd*	*Login to remote node*
logout	*End terminal session*
setenv *name v*	*Set env var to value v*
unsetenv *name1 name2...]*	*remove environment variable*

Output, Communication, & Help

Command	Description
lpr -P *printer f*	*Output file f to line printer*
script *[f]*	*Save terminal session to f*
exit	*Stop saving terminal session*
mail *username*	*Send mail to user*
biff *[y/n]*	*Instant notification of mail*
man *name*	*UNIX manual entry for name*
learn	*Online tutorial*

Process Control

Command	Description
Ctrl/c *	*Interrupt processes*
Ctrl/s *	*Stop screen scrolling*
Ctrl/q *	*Resume screen output*
sleep *n*	*Sleep for n seconds*
jobs	*Print list of jobs*
kill [%n]	*Kill job n*
ps	*Print process status stats*
kill -9 *n*	*Remove process n*
Ctrl/z *	*Suspend current process*
stop *%n*	*Suspend background job n*
command&	*Run command in background*
bg [%*n*]	*Resume background job n*
fg [%*n*]	*Resume foreground job n*
exit	*Exit from shell*

Environment Status

Command	Description
ls *[d] [f...]*	*List files in directory*
ls *-l [f...]*	*List files in detail*
alias *[name]*	*Display command aliases*
printenv *[name]*	*Print environment values*
quota	*Display disk quota*
date	*Print date & time*
who	*List logged in users*
whoami	*Display current user*
finger *[username]*	*Output user information*
chfn	*Change finger information*
pwd	*Print working directory*
history	*Display recent commands*
! n	*Submit recent command n*

File Manipulation

Command	Description
vi *[f]*	*Vi fullscreen editor*
emacs *[f]*	*Emacs fullscreen editor*
ed *[f]*	*Text editor*
wc *f*	*Line, word, & char count*
cat *f*	*List contents of file*
more *f*	*List file contents by screen*
cat *f1 f2 > f3*	*Concatenates f1 & f2 into f3*
chmod *mode f*	*Change protection mode of f*
cmp *f1 f2*	*Compare two files*
cp *f1 f2*	*Copy file f1 into f2*
sort *f*	*Alphabetically sort f*
split *[-n] f*	*Split f into n-line pieces*
mv *f1 f2*	*Rename file f1 as f2*
rm *f*	*Delete (remove) file f*
grep *'ptn' f*	*Outputs lines that match ptn*
diff *f1 f2*	*Lists file differences*
head *f*	*Output beginning of f*
tail *f*	*Output end of f*

Compiler

Command	Description
cc *[-o f1] f2*	*C compiler*
lint *f*	*Check C code for errors*
f77 *[-o f1] f2*	*Fortran77 compiler*
pc *[-o f1] f2*	*Pascal compiler*

Press RETURN at the end of each command, except those marked by an asterisk (*).

Figure 2.4: UNIX Quick Reference card p2.

Lab Activity #1 – Part II: viewing data

3.0.1 Data image examples

Three small datasets are provided. These are labeled “sonar.su,” “radar.su,” and “seismic.su” and are located in the directory

/data/Data1/

We will pretend that these data examples are “data images,” which is to say these are examples that require no further processing.

Do the following:

```
$ cd                        …(this takes you to your home directory)
$ cd  yourusername      (this takes you to /gpfc/yourusername)
```

This "$" represents the prompt at the beginning of the commandline.
Do not type the "$" when entering commands.

```
$ mkdir Temp1                (this creates the directory Temp1)
$ cd Temp1                   (change working directory to Temp1)
$ cp /data/Data1/sonar.su .
$ cp /data/Data1/radar.su .
$ cp /data/Data1/seismic.su .
                                  ^ This is a literal dot ".", which
                                  means "the current directory"
$ ls                 ( should show the file sonar.su )
```

For the rest of this document, when you are directed to make "Temp" directories, it will be assumed that you are putting these in your personal scratch working directory.

3.1 Viewing an SU data file: Wiggle traces and Image plots

Though we are assuming that the examples sonar.su, seismic.su, and radar.su are finished products, our mode of presentation of these datasets may change the way we view them entirely. Proper presentation can enhance features we want to see, suppress parts of the data that we are less interested in, accentuate signal and suppress noise. Improper presentation, on the other hand, can take turn the best images into something that is totally useless.

3.1.1 Wiggle traces

A common mode of presentation of seismic data is the "wiggle trace." Such a representation consists of representing the oscillations of the data as a graph of amplitude as a function of time, with successive traces plotted side-by-side. Amplitudes of one polarity (usually positive) are shaded black, where as negative amplitudes are not shaded. Be aware that such presentation introduces a bias in the way we view the data, accentuating the positive amplitudes. Furthermore, wiggle traces may make dipping structures appear fatter than they actually are owing to the fact that a trace is a vertical slice through the data.

In SU we may view a wiggle trace display of data via the program suxwigb. For example, viewing the sonar.su data as wiggle traces is done by "redirecting in" the data file into "suxwigb"

```
$ suxwigb < sonar.su          &
```

```
                    ^ the ampersand (&) means "run in background"
                      so you get your commandline back
```

This should look horrible! The problem is that there are 584 wiggle traces, side by side. Place the cursor on the plot and drag, while holding down the index finger mouse button. This is called a “rubberband box.” Try grabbing a strip of the data of width less than 100 traces, by placing the cursor at the top line of the plot, and holding the index finger mouse button while dragging to the lower right. Zooming in this fashion will show wiggles. The less on here is that you need a relatively low density of data on your print medium for wiggle traces.

Place the mouse cursor on the plot, and type ”q” to kill the window.

Try the seismic.su and the radar.su data as wiggle traces via

```
$ suxwigb < seismic.su &
$ suxwigb < radar.su &
```

In each case, zoom in on the data until you are able to see the oscillations of the data.

3.1.2 Image plots

The seismic data may be thought of as an array of floating point numerical values, each representing a seismic amplitude at a specific (t, x) location. A plot consisting of an array of gray or color dots, with each gray level or color representing the respective value is called an “image” plot.

If we view An alternative is an image plot:

```
$ suximage < sonar.su &
```

This should look better. We usually use image plots for datasets of more than 50 traces. We use wiggle traces for smaller datasets.

3.2 Greyscale

There are only 256 shades of gray available in this plot. If a single point in the dataset makes a large spike, then it is possible that most of the 256 shades are used up by that one amplitude. Therefore scaling amplitudes is often necessary. The simplest processing of the data is to amplitude truncate (“clip”) the data. (The term “clip” refers to old time strip chart records, which when amplitudes were too large appeared if someone had taken scissors and clipped of the tops of the sinusoids of the oscillations.) Try:

```
$ suximage < sonar.su perc=99 &
$ suximage < sonar.su perc=99 legend=1
```

The perc=99 passes only those items of the 99th percentile and below in amplitude. (You may need to look up “percentile” on the Internet.) In other words, it “clips” (amplitude truncates) the data to remove the top 1 per cent of amplitudes. Try different values of ”perc” to see what this does.

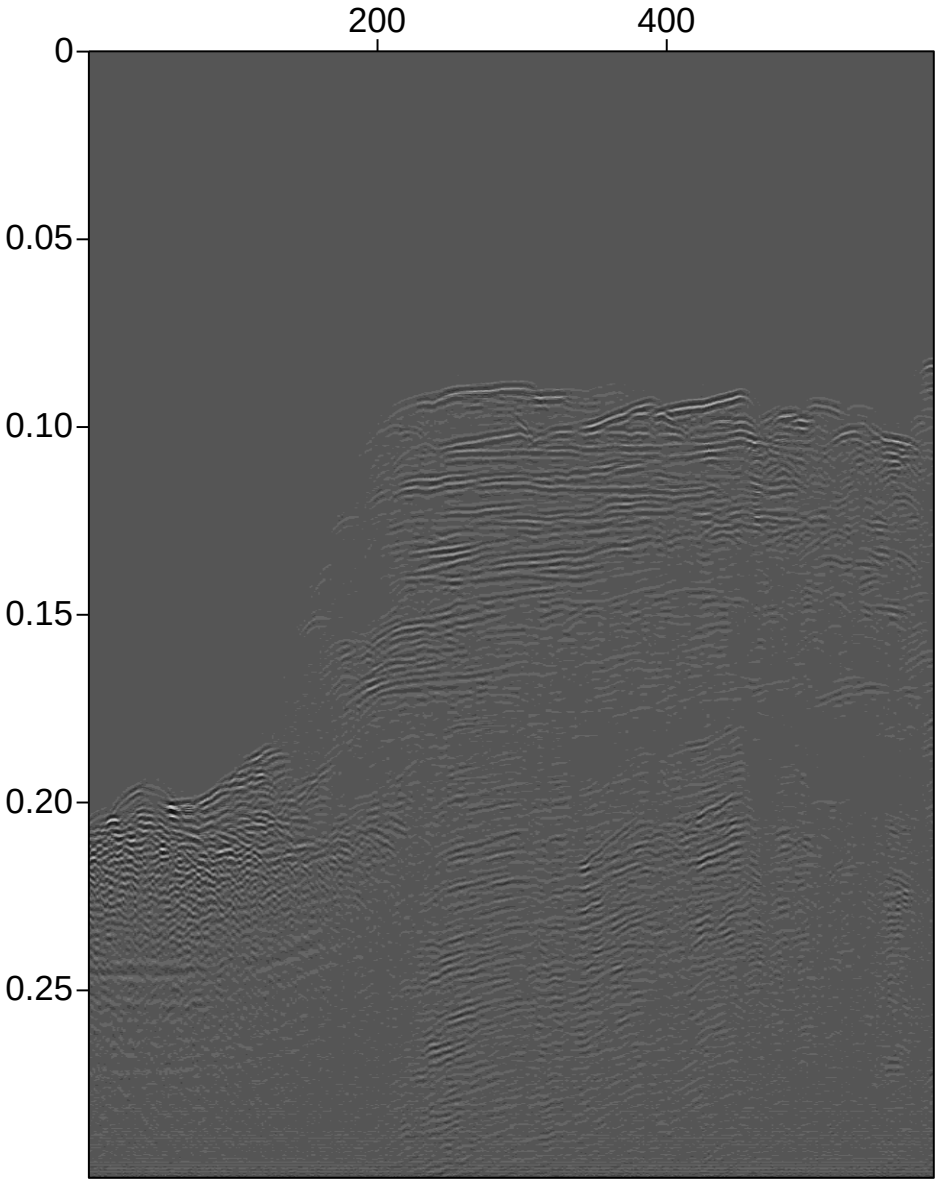

Figure 3.1: Image of sonar.su data (no perc). Only the largest amplitudes are visible.

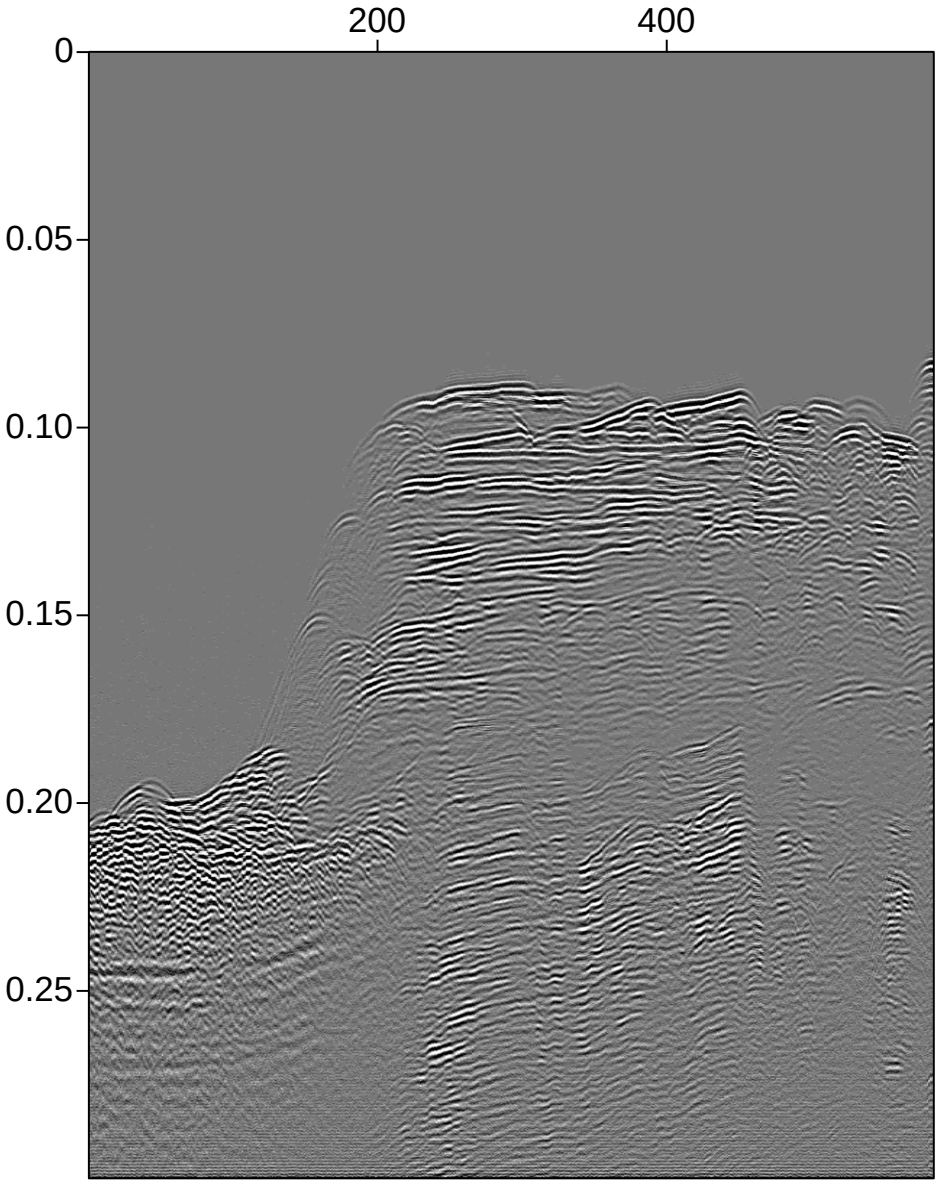

Figure 3.2: Image of sonar.su data with perc=99. Clipping the top 1 percentile of amplitudes brings up the lower amplitude amplitudes of the plot.

3.3 Legend ; making grayscale values scientifically meaningful

To be scientifically useful, which is to say “quantitative” we need to be able to translate shades of gray into numerical values. This is done via a gray scale, or ”legend”. A “legend” is a scale or other device that allows us to see the meanings of the graphical convention used on a plot. Try:

```
$ suximage < sonar.su   legend=1       &
```

This will show a grayscale bar.

There are a number of colorscales available. Place the mouse cursor on the plot and press “h” you will see that further pressings of “h” will re plot the data in a different colorscale. Now press “r” a few times. The “h” scales are scales in “hue” and the “r” scales are in red-green-blue (rgb). It is important to see that the brightest part of each scale is chosen to emphasize a different amplitude.

With colormapping some parts of the plot may be emphasized at the expense of other parts. The issue of colormaps often is one of selecting the location of the “bright part” of the colorbar, versus darker colors. Even perfectly processed data may be rendered uninterpretable by a poor selection of colormapping. This effect may be seen in Figure 3.4.

Repeat the previous, this time clipping by percentile

```
$ suximage < sonar.su   legend=1   perc=99      &
```

The ease at which colorscales are defined, and the fact that there are no real standards on colorscales, mean that effectively every color plot you encounter requires a colorscale for you to be able to know what the values mean. Furthermore, some colors ranges are brighter than others. By moving the bright color to a different part of the amplitude range, you can totally change the image. This is a source of richness of display, but it is also a potential source of trouble, if the proper balance of color is not chosen.

3.4 Display balancing and display gaining

A common data amplitude balancing is to balance the colorscale on the median values in the data. The “median” is the middle value, meaning that half the values are larger than the median value and half the data are less than the median value. Thus, the traces are normalized by this middle value.

Another possibility is to scale traces by dividing by some constant value. For example dividing each trace by the square root of the average of the sum of the square of its values (RMS).

Type these commands to see that in SU:

```
$ sunormalize norm=balmed < sonar.su | suximage legend=1
$ sunormalize norm=balmed < sonar.su | suximage legend=1 perc=99
```

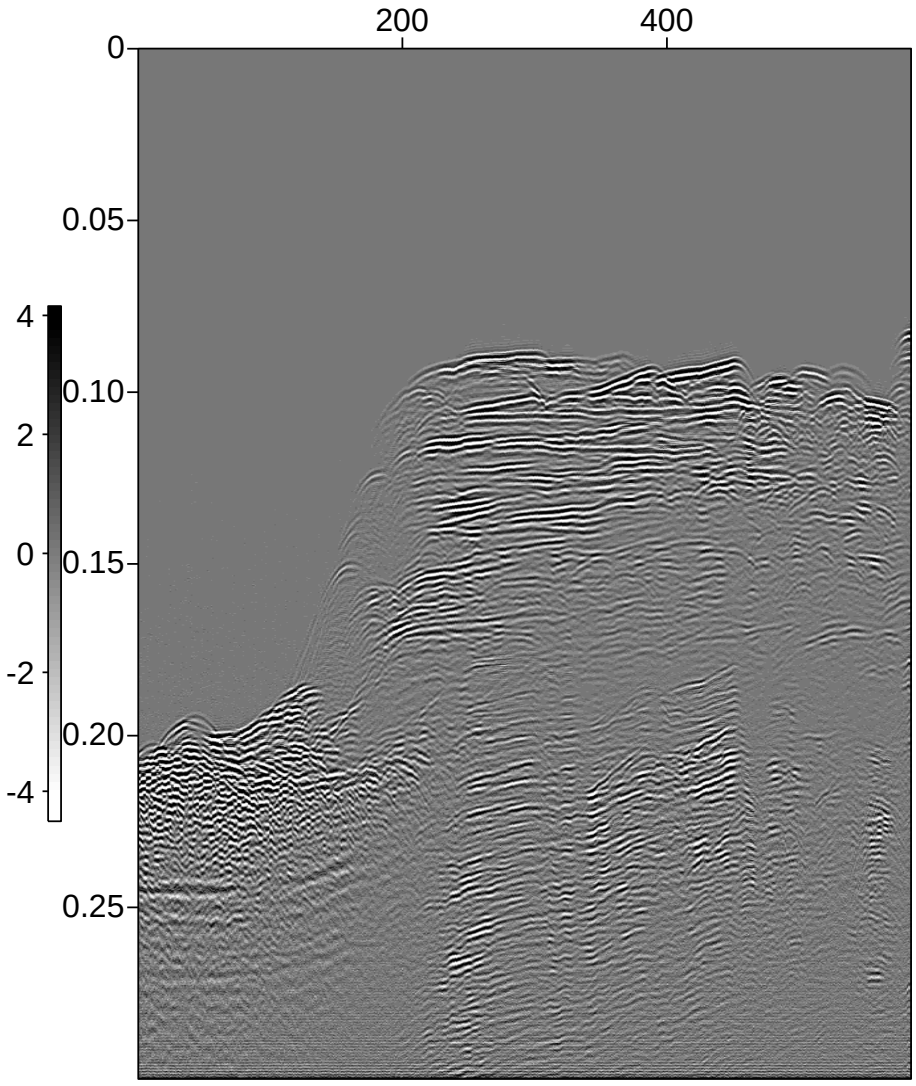

Figure 3.3: Image of sonar.su data with perc=99 and legend=1.

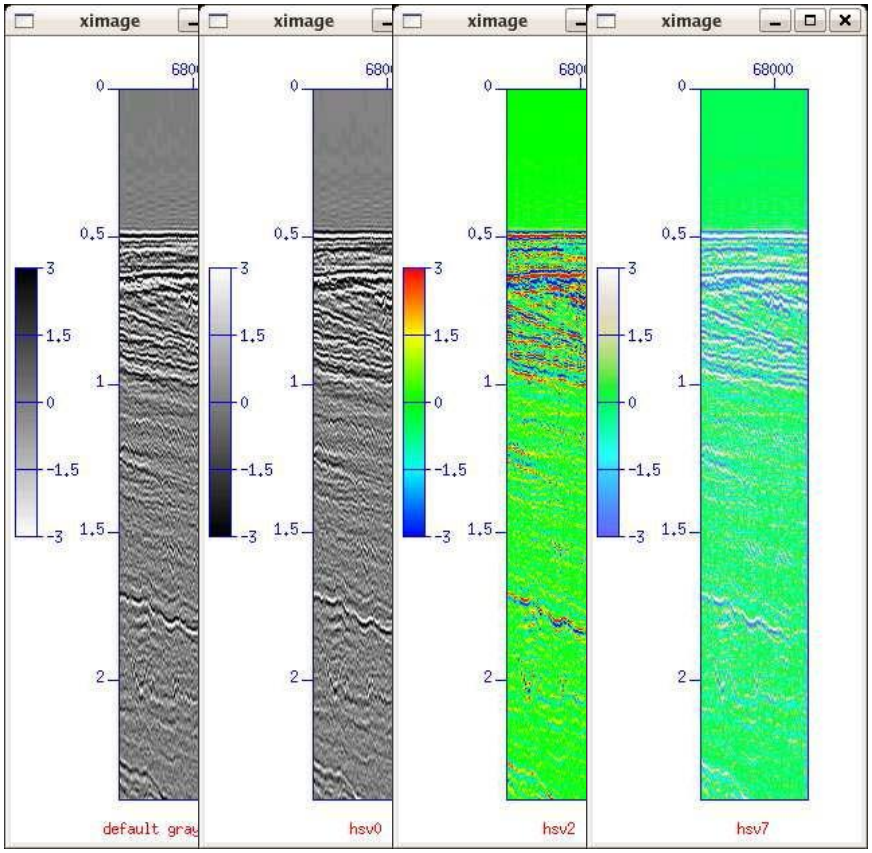

Figure 3.4: Comparison of the default, hsv0, hsv2, and hsv7 colormaps. Rendering these plots in grayscales emphasizes the location of the bright spot in the colorbar.

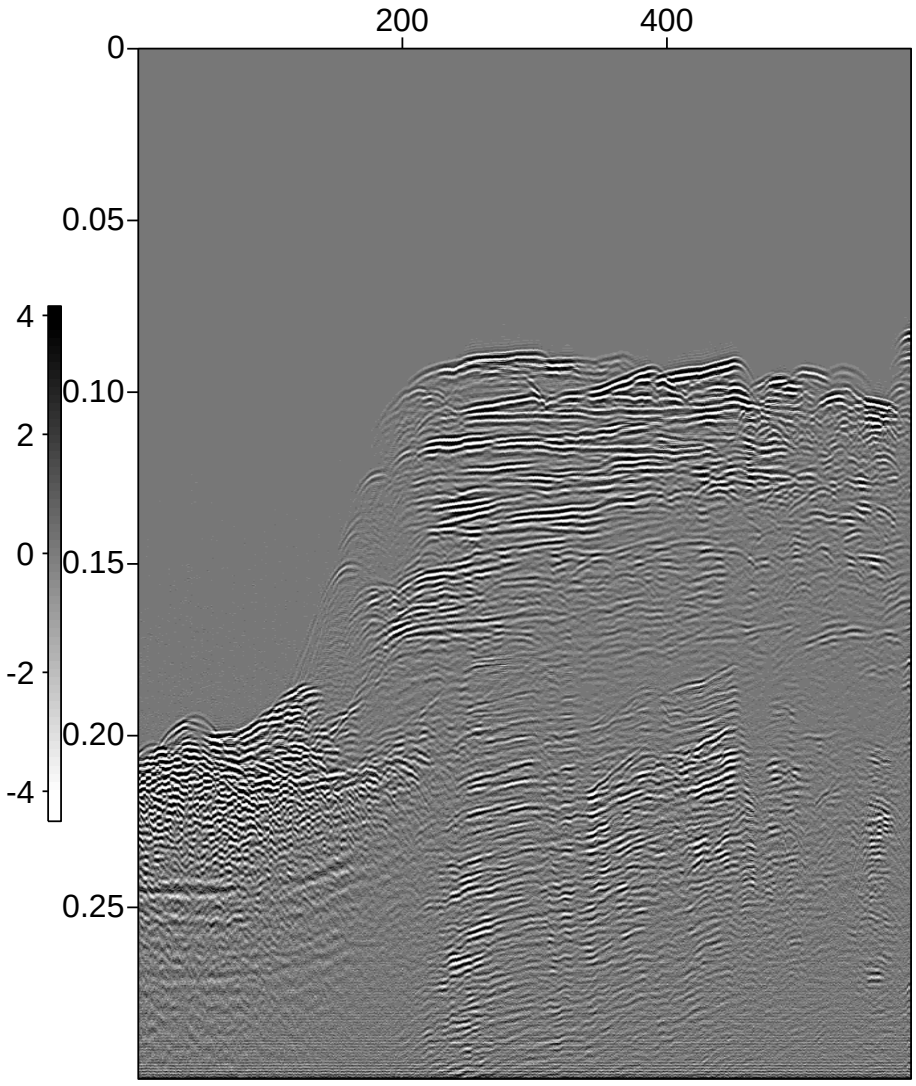

Figure 3.5: Image of sonar.su data with perc=99 and legend=1.

You may find perc=99 to be useful. You may find that you have to apply an "RMS" balancing to make the data look a bit more uniform

```
$ sunormalize norm=balmed < sonar.su |
                    sunormalize norm=rms |   suximage legend=1
$ sunormalize norm=balmed < sonar.su |
                    sunormalize norm=rms |   suximage legend=1 perc=99
```

Again, these commands are written as one long line, and are broken here to fit on the page. You may zoom in on regions of the plot you find interesting.

If you put both the median normalized and simple perc=99 files on the screen side-by-side, there are differences, but these may not be striking differences. The program suximage has a feature that the user may change colormaps by pressing the "h" key or the "r" key. Try this and you will see that the selection of the colormap can make a considerable difference in the appearance of the image. Even with the same data, the colormap.

For example in Figure 3.7 we see the result of applying median balancing. We might consider applying sunormalize directly to the seismic data

```
$ suximage < seismic.su wbox=250 hbox=600 cmap=hsv4 clip=3 title="no median" &
```

compared with applying the median balancing

```
$ sunormalize norm=balmed < seismic.su |
          suximage wbox=250 hbox=600
                    cmap=hsv4 clip=3 title="median filtering" &
```

This result looks bizarre because the traces individually have different median values and consequently have different ranges of amplitudes. An improved picture may be obtained by applying an RMS normalization to the traces after they have been median filtered via,

```
$ sunormalize norm=balmed < seismic.su | sunormalize norm=rms |
      suximage wbox=250 hbox=600
                    cmap=hsv4 clip=3 title="median filtering" &
```

In each of these examples, the line is broken to fit on the page. When you type this, the pipe | follows immediately after the seismic.su.

There are other possibilities. We may consider simply normalizing the data by the maximum or minimum value, or by some other constant. Furthermore, we have the question of whether the process be applied trace by trace, or over the whole panel of data.

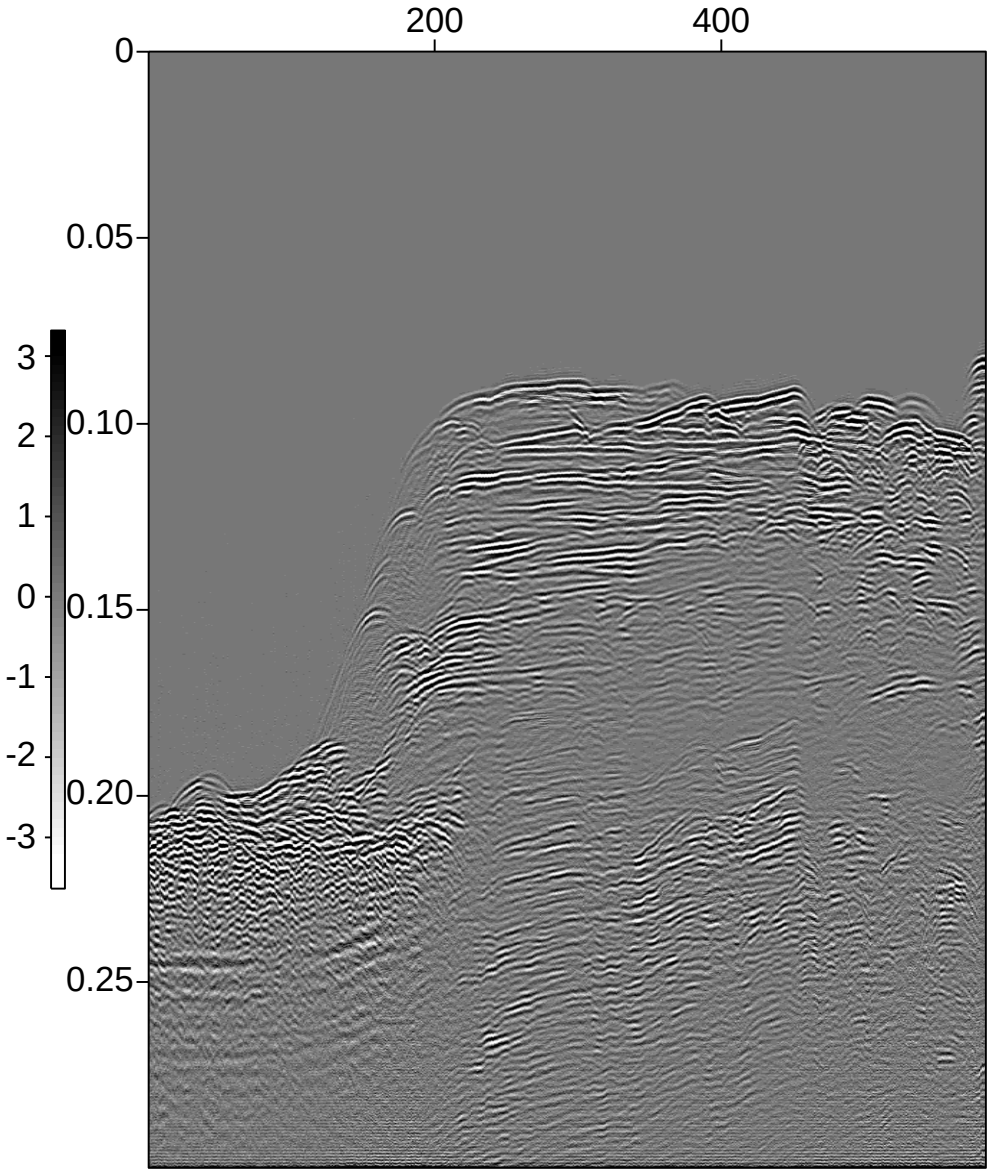

Figure 3.6: Image of sonar.su data with median balancing and perc=99

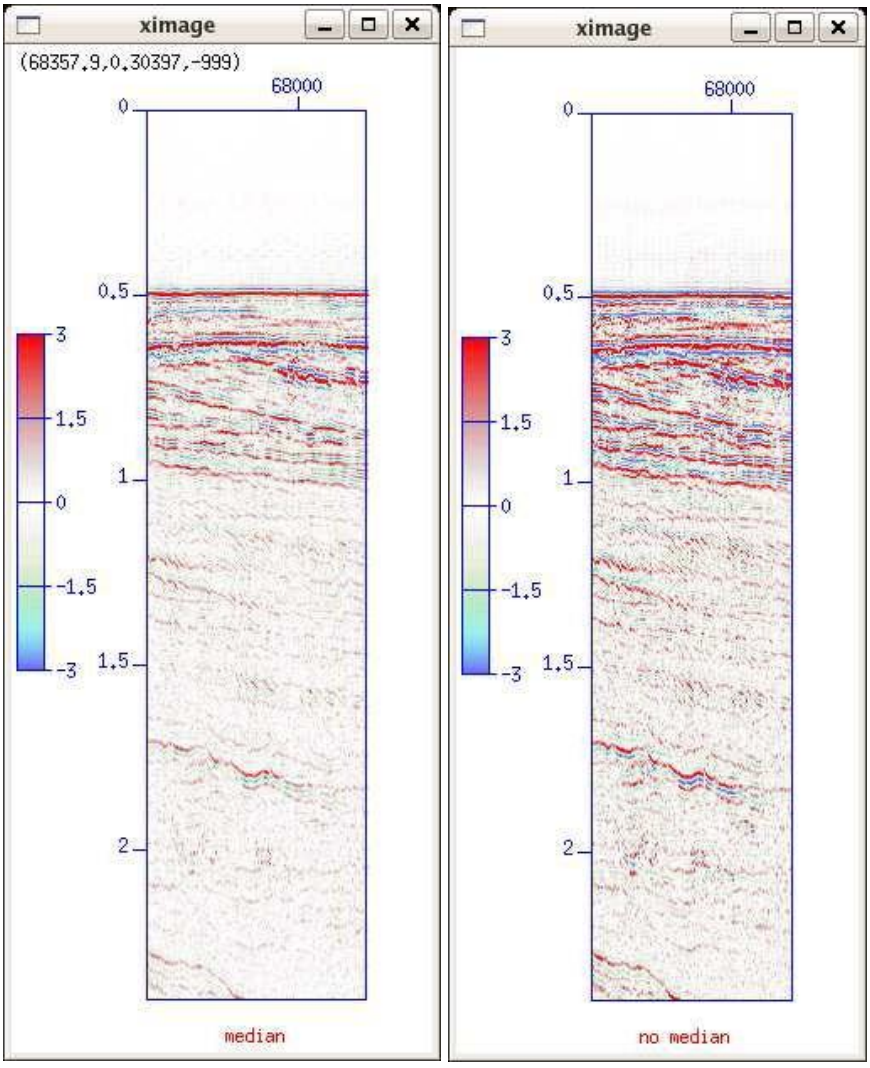

Figure 3.7: Comparison of seismic.su median-normalized, with the same data with no median balancing. Amplitudes are clipped to 3.0 in each case. Notice that there are features visible on the plot without median balancing that cannot be seen on the median normalize